PLURIONE

BITÁCORA DE DESARROLLO ÁGIL (SCRUM)

Registro de actividades y retrospectivas · corte al 17 de octubre de 2026

Plataforma de Observabilidad de Datos con Inteligencia Artificial

Prototipo académico con simulación de telemetría, monitoreo en tiempo real y asistente J.A.R.V.I.S. 2.0.

Desarrollador	Diego Rivera Aguilar
Empresa de referencia	PluriOne S.A. de C.V.
Periodo documentado	9 de septiembre al 17 de octubre de 2026
Fecha de corte	17 de octubre de 2026
Versión	Actualización posterior a la integración del Pull Request #6

Nota de alcance: El proyecto se valida como demostración académica local; los pipelines y las métricas son simulados. No se presenta como implementación productiva en infraestructuras empresariales.

Datos generales y criterio de registro

Campo	Descripción
Proyecto	PluriOne — Plataforma de Observabilidad de Datos con Inteligencia Artificial
Participación	Diego Rivera Aguilar: desarrollo y coordinación individual del backlog; roles de Product Owner y Scrum Master adaptados al ejercicio académico.
Periodo cubierto	9 de septiembre al 17 de octubre de 2026
Metodología	Organización ágil inspirada en Scrum, con planificación, seguimiento personal, revisión y retrospectiva por sprint.
Alcance	MVP académico con telemetría simulada y asistente J.A.R.V.I.S. 2.0

La bitácora preserva los cuatro periodos indicados en el documento original (hasta el 13 de octubre) y añade un quinto sprint de consolidación hasta el 17 de octubre de 2026. Las actividades finales se agrupan por periodo cuando no consta una fecha individual; no se presenta el seguimiento personal como reuniones formales de un equipo Scrum.

Sprint	Fechas	Foco principal
1	09–22 septiembre	Requisitos, PRD/MVP y PostgreSQL.
2	23 septiembre–04 octubre	FastAPI, modelo relacional, JWT, SMTP.
3	05–10 octubre	React, Recharts, WebSocket, J.A.R.V.I.S. inicial.
4	11–13 octubre	Docker Compose, GitHub y documentación inicial.
5	14–17 octubre	Mejoras J.A.R.V.I.S. 2.0, seguridad, QA, recuperación y PR #6.

Sprint 1. Fundamentación, requisitos y base de datos

Fechas: 9 al 22 de septiembre de 2026.

Objetivo del sprint. Establecer la problemática y el alcance del MVP, seleccionar las métricas de monitoreo y configurar un modelo relacional de persistencia.

Product Backlog y planificación

- Redactar el Documento de Requisitos del Producto (PRD) y el Producto Mínimo Viable (MVP).
- Definir los pipelines representativos del simulador, las métricas y los estados operativos.
- Preparar PostgreSQL y las entidades de usuario y anomalía.

Desarrollo y tareas realizadas

- Configuración de una base PostgreSQL en el entorno local y prueba inicial de conectividad.
- Modelado de users con nombre, correo, contraseña hasheada, activación de cuenta y código de verificación.
- Modelado de anomaly_logs con referencia al usuario, pipeline, tipo de métrica, valor, estado, recomendación y marca de tiempo.
- Priorización de los indicadores de latencia (ms), calidad de datos (%) y fallos (%).

Daily Scrum y seguimiento

Seguimiento individual del alcance y de la consistencia entre PRD, MVP y campos almacenados. Se revisó la necesidad de relacionar anomalías con el usuario autenticado para evitar consultas cruzadas.

Sprint Review y retrospectiva

Resultado: base conceptual, documentos de alcance y estructura relacional inicial. Lección aprendida: diferenciar escenarios de pipelines simulados de una fuente empresarial real y establecer desde el inicio la trazabilidad de cada registro.

Evidencia documental Se conserva el planteamiento del sprint original. No se atribuyen a este periodo pruebas exhaustivas ni despliegues de producción.

Sprint 2. Backend, API REST y seguridad

Fechas: 23 de septiembre al 4 de octubre de 2026.

Objetivo del sprint. Implementar FastAPI, la persistencia y los mecanismos de identidad necesarios para acceder a las rutas protegidas de forma controlada.

Product Backlog y planificación

- Habilitar registro de usuario, verificación SMTP, inicio de sesión y recuperación de contraseña.
- Crear operaciones de consulta de anomalías y conexión persistente con PostgreSQL.
- Emitir JWT y separar operaciones públicas de operaciones protegidas.

Desarrollo y tareas realizadas

- Construcción de FastAPI con modelos y esquemas Python, uso de SQLAlchemy y dependencias de base de datos.
- Almacenamiento de contraseñas mediante hash y emisión de tokens JWT al iniciar sesión.
- Registro inicial de cuentas con is_active=False y activación con código de seis dígitos enviado mediante SMTP.
- Creación de endpoints para solicitar recuperación y restablecer contraseña mediante código.
- Implementación de consultas de registros y de las bases de autorización por usuario.

Daily Scrum y seguimiento

Se dio seguimiento individual a la secuencia registro → verificación → inicio de sesión y al almacenamiento del estado activo. Se evaluaron errores de entrada en los códigos y los mensajes de respuesta de FastAPI.

Sprint Review y retrospectiva

Resultado: infraestructura básica de API e identidad. La evidencia de pruebas de acceso no autorizado y recuperación completa se consolidó posteriormente en el Sprint 5; no se presenta como una prueba automatizada ya ejecutada en el Sprint 2.

Sprint 3. Frontend, WebSockets e inteligencia artificial

Fechas: 5 al 10 de octubre de 2026.

Objetivo del sprint. Diseñar la navegación en React, mostrar métricas simuladas en un dashboard y establecer el canal de eventos y un asistente de diagnóstico inicial.

Product Backlog y planificación

- Diseñar las pantallas de login, registro, verificación y recuperación de acceso.
- Preparar las tres visualizaciones y el historial de anomalías.
- Integrar WebSockets y un primer panel conversacional J.A.R.V.I.S.

Desarrollo y tareas realizadas

- Interfaz React con React Router, Axios, navegación entre pantallas y estilo propio de PluriOne.
- Gráfica lineal de latencia, barras de calidad y dona de tasa de fallos con Recharts.
- Canal /ws/anomalias para recibir eventos del backend y mostrar alertas sin recarga.
- Panel J.A.R.V.I.S. para diagnóstico orientativo y cuadro flotante de bienvenida para explicar las gráficas.
- Historial y exportación a PDF mediante jsPDF y jspdf-autotable.

Daily Scrum y seguimiento

Ajustes a validación de formularios, presentación del código de seis dígitos y disposición de tarjetas de monitoreo. Se atendió la legibilidad de las vistas y la continuidad de las alertas durante la navegación.

Sprint Review y retrospectiva

Resultado: interfaz navegable e integración inicial del monitoreo en tiempo real. Lección aprendida: los campos de formulario, la autenticación de eventos y el diseño de paneles deben probarse juntos para asegurar una experiencia completa.

Sprint 4. Contenedores, control de versiones y documentación inicial

Fechas: 11 al 13 de octubre de 2026.

Objetivo del sprint. Formalizar la ejecución local, consolidar los documentos técnicos iniciales y respaldar los cambios en un repositorio remoto.

Product Backlog y planificación

- Preparar el Dockerfile y Docker Compose para PostgreSQL y FastAPI.
- Organizar los entregables README, PRD y MVP.
- Respaldar los avances con Git y GitHub mediante ramas y commits.

Desarrollo y tareas realizadas

- Configuración y validación inicial del arranque de servicios contenedorizados.
- Estructuración del repositorio con backend/, frontend/, docs/ y archivos de configuración.
- Correcciones de funcionamiento local y preparación de documentación sobre instalación y uso del prototipo.

Daily Scrum y seguimiento

Seguimiento al estado de contenedores, comandos de Git y congruencia inicial entre requisitos y funcionalidades existentes.

Sprint Review y retrospectiva

Resultado: versión preliminar funcional respaldada y documentada. La revisión posterior identificó tareas pendientes de calidad y conexión del formulario de recuperación; por ello, el cierre del 13 de octubre se registra como hito preliminar y se abre una quinta iteración.

Sprint 5. Integración de J.A.R.V.I.S. 2.0, seguridad y cierre técnico

Fechas: 14 al 17 de octubre de 2026.

Objetivo del sprint. Corregir observaciones pendientes del MVP, reforzar el aislamiento de usuarios, mejorar J.A.R.V.I.S. y comprobar la recuperación de acceso y la integración final en GitHub.

Product Backlog y planificación

- Revisar el código, los archivos ignorados por Git y la ausencia de secretos comprometidos en el listado de archivos versionados.
- Restringir anomalías, WebSockets y herramientas de IA a la identidad autenticada.
- Habilitar consultas SQL controladas desde J.A.R.V.I.S. 2.0 y mejorar el formato del chat.
- Completar el ciclo de solicitud de código y restablecimiento de contraseña desde React.
- Verificar compilación, dependencias, Git y publicación del Pull Request #6.

Desarrollo y tareas realizadas

- Se creó backend/app/jarvis_tools.py para la herramienta consultar_anomalias_usuario. El backend fija user_id según el JWT y permite consultas de hasta 10 registros recientes, con filtros por estado.

- Se integró el modelo openai/gpt-oss-20b por medio de Groq, con llamadas controladas a herramientas y respuestas técnicas en español; no se habilitaron comandos de remediación automática.
- Se implementó visualización Markdown y tablas en el chat de React, desplazamiento interno y botón «↓ Último». Se ajustó la distribución para evitar que las tablas deformaran el dashboard.
- Se completó el timestamp de cada anomalía transmitida por WebSocket, de modo que el feed pudiera presentar las horas registradas.
- Se comprobó la protección de rutas sin JWT y se corrigió get_current_user para rechazar también las cuentas inactivas.
- Se conectó la pantalla Recuperar Acceso con /solicitar-recuperacion/ mediante el botón «Enviar código de recuperación» y se probó la actualización posterior de la contraseña.
- Se ejecutaron compilaciones de Python y de React, verificación de espacios con git diff --check y auditoría de dependencias npm; el último registro de npm audit arrojó 0 vulnerabilidades.
- Se conservaron respaldos al explorar mejoras opcionales para los códigos SMTP; se retiraron esas modificaciones del código integrado para respetar el alcance de la entrega.
- Se actualizó README principal, README del frontend, PRD y MVP; se completó la fusión de PR #6 a main y se sincronizó la rama local.

Seguimiento por periodos y evidencias

Periodo / hito	Acción documentada	Evidencia concreta
14–16 octubre	Revisión de integración técnica, observaciones y necesidades de documentación.	Trabajo de iteración en rama fix/jarvis-groq-usage; actividades agrupadas por periodo.
17 octubre: Groq y consultas	Integración de J.A.R.V.I.S. 2.0, aislamiento por usuario y presentación de Markdown.	Commits 0d1a291 y 137f4ee; pruebas manuales de conversación.
17 octubre: controles JWT	Rechazo de tokens pertenecientes a cuentas inactivas.	Commit e3534da; prueba HTTP 401 con JWT de cuenta inactiva.
17 octubre: recuperación	Botón de solicitud SMTP, código recibido, cambio y nuevo inicio de sesión.	Commit b5f8ff8; confirmación en navegador.
17 octubre: pruebas generales	Compilación de Vite, chequeo Git, audit npm y arranque Docker.	1453 módulos; 0 vulnerabilidades reportadas; servicios api/db Up.
17 octubre: integración	Fusión del Pull Request #6 y actualización de main local.	Commit merge a1aef19; git status: main...origin/main.

Pruebas de aceptación manual realizadas

Caso	Evidencia	Resultado
Consultar historial sin JWT	GET /anomalias/	HTTP 401.
Invocar chat sin JWT	POST /chat/	HTTP 401.

Acceso con cuenta inactiva	JWT emitido para cuenta con is_active=False	HTTP 401.
Solicitar recuperación	Formulario React → SMTP	Código recibido.
Actualizar contraseña	Código más reciente + contraseña nueva	Cambio aceptado; acceso posterior al dashboard.
Dashboard en navegador	Telemetría simulada vía WebSocket	Gráficos, feed y tiempos observados.
Cierre de rama	PR #6 y Git local	Merged; main sincronizada.

Impedimentos y decisiones técnicas

- Error de código incorrecto: se identificó que solicitar códigos sucesivos reemplaza el anterior; el último código permitió completar el flujo.
- Vite detenido: al abrir la ruta local se obtuvo «conexión rechazada»; el frontend volvió a mostrarse tras reiniciar npm run dev.
- Presentación de Markdown: las tablas extensas afectaban el ancho de la pantalla; se limitaron al contenedor del chat.
- El simulador no incluía timestamp en el mensaje WebSocket; se agregó el campo al evento.
- La cuota y la disponibilidad de Groq son dependencias externas que pueden producir errores temporales.
- No se publicó la propuesta exploratoria de caducidad de códigos SMTP: requería cambios de base de datos y nuevas pruebas no obligatorias para la entrega.

Sprint Review y retrospectiva

Resultado del sprint. Se consolidó el prototipo académico mediante el Pull Request #6. Quedaron probados los rechazos de acceso sin autorización, la recuperación completa y la presentación del dashboard. La herramienta de J.A.R.V.I.S. quedó limitada a la información del usuario autenticado y a consultas por estado.

Lecciones aprendidas. Es indispensable validar de extremo a extremo una funcionalidad; tener un endpoint de recuperación en FastAPI no garantiza que el usuario pueda solicitar el código desde React. Asimismo, los controles del JWT deben considerar el estado activo de la cuenta, y la documentación debe describir únicamente capacidades comprobadas.

Registro de entregables y commits relevantes

Referencia	Descripción
0d1a291	Ajustes de J.A.R.V.I.S. y herramientas seguras de anomalías, en rama de trabajo.
137f4ee	Integración de J.A.R.V.I.S. 2.0 y documentación técnica.
e3534da	Rechazo de accesos JWT correspondientes a cuentas inactivas.

b5f8ff8	Botón React para solicitar código de recuperación.
PR #6 / a1aef19	Fusión de cuatro commits en main, el 17/10/2026.
README / PRD / MVP	Documentación existente revisada; edición final adicional debe publicarse y verificarse en GitHub.
Bitácora e informe	Actualización documental con corte al 17 de octubre de 2026.

Backlog posterior y observaciones de cierre

El MVP se encuentra integrado para demostración académica con datos simulados. La finalización académica al 100 % no puede certificarse sin contrastar la versión publicada con la lista íntegra de observaciones del profesor. Este punto no altera la evidencia del merge y de las pruebas registradas.

- Pendiente recomendado: pruebas automatizadas de seguridad y regresión, y verificación formal de restauración de un respaldo de base de datos.
- Pendiente de endurecimiento: vencimiento e intentos limitados para códigos SMTP, con migración versionada y ensayos específicos si se decide ampliar el alcance.
- Fuera del MVP: conectores reales de pipelines, alarmas empresariales y acciones automáticas sobre sistemas productivos.
- Pendiente documental de entrega: publicar los cambios adicionales de README, frontend/README.md, PRD, MVP, esta bitácora y el informe, verificando un commit nuevo y la sincronización con GitHub.

Cierre de bitácora. A 17 de octubre de 2026, la iteración técnica del prototipo PluriOne cuenta con un historial de desarrollo y resultados reproducibles de las principales funciones locales. La presente bitácora complementa al Informe Técnico y conserva explícitamente las limitaciones de la demostración.

Documentación y evidencias de referencia

- Bitácora de Desarrollo Ágil (Scrum) - Proyecto PluriOne.pdf, versión original, 09/09–13/10/2026.
- Informe Técnico Plataforma de Observabilidad.pdf, documento base de diez fases.
- Archivos README.md, frontend/README.md, docs/PRD.md y docs/MVP.md del proyecto.
- Repositorio: https://github.com/Diego-riveera02/G19X-TESE-DRA-122-ACADEMIC. Pull Request #6 (Merged), commit a1aef19.
- Comandos, salidas de PowerShell, pruebas manuales y capturas compartidas durante la consolidación técnica.